Threads and Parallel Design Patterns

Parallelism Design Patterns and Threads

Learning Objectives:

- Be able to use fork-join parallelism to solve a problem
- Be able to define what a thread is and explain how their costs compare to processes.
- Be able to reason about the possible outcomes under uncontrolled thread scheduling.

Complementary Reading: *Operating Systems: Three Easy Pieces (OSTEP)* — **Chapter: Threads Intro** and **Chapter: Thread API** (pthreads).

Parallel Design Patterns

- We know how to use `fork` and `wait` to create parallelism within a program.
- Example problem: Calculate the frequency of each word in a set of documents
    - Steps:
        - Read each document
        - Tokenize each document to determine words
        - Count the number of times each token occurs in each document
        - Merge the results.
    - Possible parallel design: Create a separate process for each document.
    - Possible parallel design: Create a separate process for each step.
- **Parallel design patterns** provide a structure to exploit parallelism in a problem.
    - High-level abstractions describe where parallel work exists in the problem
        - Data Parallelism: split the data into distinct units and process each unit parallel
        - Task Parallelism: split the process into distinct tasks and process tasks parallel
        - Sometimes both! (e.g., map-reduce)
    - Programming models give structured primitives to implement a problem's parallelism:
        - Fork-join parallelism (common in data parallelism)